Design decisions put to the owner

Every decision here was a fork the evidence could not settle on its own: either it changed the architecture, or it traded cost against reliability. Each entry gives the question as it was asked, the options offered, what was chosen, and why the question existed at all.

Dates are 2026. The standing rule from 22 September — **safety and reliability before cost, size and convenience** — is what several of these answers turn on.

---

1. The harness connector

Asked: the specification (§7, deviation 1) called for the OEM 94-way connector to be replaced by functionally-split sealed connectors plus an adapter harness. The physical part had since been identified from photographs of the machine. Keep the deviation, or put the OEM connector on the board?

Chosen: the OEM Bosch 94-way on the board, no adapter. Spec deviation 1 is recorded as reversed.

Consequence: the board-side land pattern is now load-bearing and has to come from a Bosch distributor. Neither part number returns a public catalogue hit.

---

2. Four part-selection findings, approved together

These four were presented as recommendations and approved with one "ok" on 22 September. Each one came out of choosing a part and discovering the drawn circuit could not accept it.

2a. Where the injector and EGR current comes from — POWER-PATH

The problem: every high-current load (injector hold ≈10 A, EGR up to 6.4 A stall, relay coils) draws through connector pin 21, a 20 A fuse, the EMI filter and a reverse-battery FET. The reverse-battery block sized that FET for **5 A**, and the EMI filter block never stated a DC current at all. A 22 µH inductor that does not saturate at 15–20 A is a large part. Meanwhile pins 04 and 06 are separate battery feeds, fused 10 A each in the loom, that this board uses only to measure voltage.

Options: move the injector and EGR supply onto pins 04/06 with their own protection, leaving pin 21 for the electronics; or keep everything on pin 21 and resize the fuse, filter and reverse FET for the full load,

with one contact carrying it.

Chosen: move the high-current loads to pins 04/06. Status: built 8 October 2026 (`power_path.cir` , power_input sheet).

How it was built took three more questions, recorded in §5.

2b. Boost converter stability while cranking — BOOST-SLOPE

The problem: the injector boost controller (TPS40210-Q1) adds a fixed internal slope-compensation ramp. Its Equation 9 caps the current-sense resistor at `VDD·L·fSW / (60·(VOUT + VD - VIN))` , and 80% of that is the recommended value. The drawn 82 mΩ passes at 13.5 V (ceiling 99.7 mΩ) and **fails at the 6 V cranking corner** (ceiling 44.9 mΩ, so 1.83× over): the converter goes subharmonic exactly when the engine is being started. Worse, a 330 µH inductor that saturates above the 2.2 A current limit is not a part that exists in surface mount — the search found 330 µH parts rated 0.2–0.4 A — and the buildable direction, a smaller inductor, lowers the ceiling further (100 µH allows 30 mΩ).

Options: feed the controller's supply pin from the output side, which is TI's own remedy and steepens the ramp, and use 100 µH; or accept subharmonic operation while cranking.

Chosen: feed VDD from the clamped output side, 100 µH. Revised 8 October: 150 µH (Würth 7447709151), see §8. Status: built 8 October 2026.

2c. Sensor-supply protection — SENSOR-PTC

The problem: the requirement asked for a resettable fuse of "R25 = 3.0 Ω nominal, −30% spread". Real PTCs are not specified that way: Bourns MF-USMF020 gives a **range, 0.40–5.00 Ω**, and at its 0.40 Ω minimum it contributes 14% of the series resistance the fault-current ceiling assumed. Its hold current also halves from 0.20 A at 23 °C to 0.10 A at 85 °C, against a ≥200 mA requirement.

Chosen (first pass): a fixed series resistor sets the fault ceiling and the fuse only trips. This was later superseded — see decision 4.

2d. The 3.0 V and 3.3 V input clamps — LOWV-CLAMP

The problem: fifteen zeners clamp the analog, speed and discrete inputs. They were specified at 5 mA but sit at tens of microamps, where a sub-5 V zener's knee is soft. The datasheet allows **10 µA of leakage at only 1 V.** On a discrete input, 10 µA through the ~28 kΩ source is 0.28 V against a logic-high margin of 0.36–0.41 V while cranking; on an analog input it is a signal-dependent error on the ADC. The simulation's idealised sharp-knee zener hid all of it.

Chosen: replace them with a low-leakage clamp to the 3.3 V rail (BAV199). Status: done for the analog and cam inputs; the discrete inputs needed a different answer — see decision 3.

---

3. The discrete switch inputs

Why it came back: BAV199 works where a clamp only conducts on a fault. The five switch inputs clamp **continuously**, because a 6–40 V input cannot be divided into logic range otherwise: reading a logic high at 6 V needs a divider ratio above 0.385, and staying under 3.3 V at 40 V needs one below 0.0825. Those do not overlap. Three candidate clamps were checked and all three fail:

Candidate	Why it fails
3.0 V zener (BZX84-C3V0-Q)	soft knee at µA; the leakage that started this
Rail clamp (BAV199-Q to 3V3)	parks the pad at 3.9 V continuously — beyond the pin's rating, as an operating point rather than a fault
Shunt reference (LM4040-Q1 3.0 V)	draws ~35 µA of bias below breakdown (TI SNOS633N, Fig. 5-4); the cranking-dip input can supply only 20 µA, so the node sags to ~2.6 V with no guaranteed bound, and it fails outright on the pull-up channels

Options offered: an NPN buffer per input; a stronger drive plus the LM4040, with special cases for the two contact-to-ground channels; or a dedicated automotive switch-detection IC on SPI.

Chosen: the NPN buffer. 10 kΩ into the base, 4.7 kΩ and 4.7 µF to ground, a BAV199-Q diode holding the base against reverse battery, and the transistor's collector as the MCU pin with a 10 kΩ pull-up to 3.3 V. The pad now only ever sees 0–3.3 V, and nothing on it has to conduct continuously.

What the safety rule changed inside this decision: the input resistor was first drawn at 100 kΩ, which draws 0.11 mA through a closed contact at 12 V. Non-gold switch and relay contacts at that current oxidise into an intermittent open — a field failure, not a bench one. 10 kΩ gives 1.1 mA. Everything is checked at the worst corner rather than typical: hFE 30 (datasheet minimum 60, halved for −40 °C) still pulls the pad to 0.03 V from a 6 V input, and a typical part at 125 °C stays off below a 1.3 V input.

Cost: the logic is inverted — pad low means input active — which is recorded in the pin map, and the failure modes (transistor open reads inactive, collector-emitter short reads active) are handed to the firmware plausibility spec, because one polarity cannot make every channel fail safe.

Status: done.

4. The 5 V sensor supply

Why it came back: the fix approved in 2c does not survive the real source impedance. The sensor rail block modelled the 5 V supply as ideal behind 0.1 Ω, but it comes from a **1 A buck**. So:

- a shorted sensor harness hits the buck's current limit long before any fuse trips; 5V_MAIN collapses, 3V3_MCU follows it, and the supervisor shuts the engine down. One chafed wire stops the machine.
- a sensor supply wire shorted to **battery** back-feeds 24 V into 5V_MAIN through the fuse. No series resistor or PTC prevents that.

Options offered:

Option	What it buys	What it costs
Tracker per group + overvoltage cut-off	each group survives a short to ground *and* to battery; other groups and the MCU are unaffected	about 8 more parts
Series resistor + PTC (as approved in 2c)	cheapest	a harness short browns out the MCU; a battery short back-feeds the rail
Trackers fed directly, with a lower main TVS	fewer parts	rewrites the whole board's transient design, and needs the alternator question (U16) answered first

Chosen: a tracking regulator per sensor group (TPS7B4253-Q1, whose output survives −1 to +45 V and limits at 301–520 mA, tracking 5V_MAIN to ±4 mV) behind a 100 V-class overvoltage cut-off.

Why the cut-off is needed: the tracker's input pin is rated 45 V and this board's battery rail reaches 73.3 V for 50 µs on ISO 7637-2 pulse 2a. The cheap alternatives were worked through and discarded: a series resistor with a TVS must clamp below 45 V, so it would conduct through the entire 400 ms load dump (about 7 J), and any series resistance breaks the 0.52 V of dropout headroom at a 6 V cranking input; an emitter or source follower loses its own Vbe/Vgs at that same corner. A cut-off instead disconnects for the 50 µs pulse, while a 47 µF capacitor holds the sensors up (0.16 V of droop), and stays connected through the 40 V dump.

Status: built 8 October 2026. The cut-off controller is the **TPS48110-Q1** (see §7). `sensor_rail.cir` was rewritten around it.

5. How the POWER-PATH feeds are built (8 October)

§2a decided that the loads move to pins 04/06. These three questions decided how. Each was asked with a recommended option, and each was answered with it.

5a. Which loads go on which pin

Asked: how should the high-current loads be split across pins 04 and 06?

Options: - **04 injector, 06 EGR** (recommended). Pin 04 feeds the injector hold switches and the boost converter input; pin 06 feeds the EGR H-bridge. Separate rails, separate protection: an EGR short blows only its own fuse, and no two fuses are paralleled. The injector averages well under its fuse rating (10 A only during holds of about 1–3 ms), and the EGR's 6.4 A stall is inside its 10 A fuse. - **04 + 06 paralleled** onto one high-current rail. Two fuses in parallel never share evenly: one blows first and dumps the whole load on the other.

Chosen: 04 injector (`VBAT_INJ`), 06 EGR (`VBAT_EGR`).

Why it was a question: U7, whether the OEM uses pins 04/06 as load feeds at all, is still open. What makes them usable is the harness itself: each pin has its own 10 A loom fuse, and a fuse is sized to protect its wire.

5b. Reverse-battery protection on the new rails

Asked: what should provide reverse-battery protection on the two new rails?

Options: - **A P-FET per rail** (recommended). Same passive topology as Q1 on pin 21: a gate resistor to ground plus a zener, and no controller IC, so nothing has to be rated for the +73 / −48 V swing. It needs a lower-RDS(on) part, chosen from a retrieved datasheet, for 6.4 A continuous at a 105 °C board. The reversed-battery fault is removed rather than managed. - **Fuse only.** No series element: a reversed battery drives current through the bridge FETs' body diodes until the 10 A loom fuse blows. No drop or heat in normal running, but it relies on the FETs surviving the fuse's clearing time — a managed failure, which the standing rule argues against.

Chosen: a P-FET per rail — SQM120P10-10m1L (Vishay doc 76943, retrieved 8 October).

Why it was a question: pin 21's SQD50P08-25L could not simply be copied. At the EGR's 6.4 A stall it would dissipate 1.3–1.8 W, which takes a DPAK on a 105 °C board past its junction limit. The chosen part runs at 139 °C (155 °C while cranking), checked in `power_path.cir`.

5c. What stays on pin 21

Asked: which loads stay on pin 21, the filtered electronics feed?

Options: - **Buck, relays, metering** (recommended). Pin 21 keeps the buck (all the electronics), the relay coils (~84 mA) and the metering valve's freewheel diode (0.86 A max; the valve itself is powered by the

OEM's 20 A fuse 8F1). The boost controller's VDD stays on VBAT_PROT until BOOST-SLOPE moves it. Pin 21 then carries a couple of amps. - **Buck only.** Move the relay coils and the metering freewheel onto the new rails as well. Pin 21 becomes electronics-only, but the relays and the metering valve then depend on the 04/06 fuses too.

Chosen: buck, relays and metering stay on pin 21.

Consequence: L101, L102 and F101 are unblocked and are now sized for pin 21's reduced load, which still has to be stated as a number.

---

6. The toolchain on a new machine (7–8 October)

Setting the project up on macOS raised three questions. They are about tools, not the circuit, but two of them decide which simulator and which libraries the circuits are checked against.

6a. Which KiCad

Asked: the project was generated and verified with kicad-cli 7.0.11, but Homebrew offers only KiCad 10. Install 7.0.11 from kicad.org, KiCad 10 from Homebrew, or skip KiCad for now?

Options: 7.0.11, which matches the verified netlists (recommended at the time); KiCad 10, which is easier to keep updated, but its netlist export and libraries may differ; or no KiCad.

Chosen: KiCad 10 (10.0.7, installed by the owner).

Consequence: two things had to change for KiCad 10. `kicad-cli` 10 writes its netlist one token per line, so the parsers in `check_netlist.py` and `assign_passives.py` were changed to accept either layout. The results were identical to KiCad 7's, and `passive_fp.json` came out byte for byte the same. The generators copy library symbols into each sheet, so they are pointed at **KiCad 7.0.11's symbol library**, downloaded separately. With it, an unchanged sheet regenerates identically apart from UUIDs. KiCad 10's library would have changed every symbol on every regenerated sheet. That library choice was not put to the owner; it follows from this answer.

6b. Which ngspice

What happened: Homebrew's ngspice 47 failed three blocks that pass on the original machine. `injector_turnoff` and `boost_converter` never finished, and `metering_unit_pwm` missed by 3 %. The suite also stalled at random. Fixes for all three were found: a ramp on an on/off charger model, a step-size cap and a time-weighted average. But the random stall remained.

The owner's position: first, that the circuits must work on the latest version too. Then, after the random stall: build ngspice 42 from source rather than spend more time on 47.

Chosen: ngspice 42, built from source into `~/opt/ngspice-42`. The committed suite passes 24/24 on it unchanged (25/25 with `power_path`).

Still open: the suite does not pass on ngspice 47 for the three causes above. Upgrading the simulator is a change that needs the whole suite re-run (`docs/HANDOFF.md` §2 and §5).

6c. Keep the ngspice 47 fixes or not

Asked: with 42 passing the committed code, revert the 47-specific edits to the netlists, `run_sim.py` and `ngrun.py`, and commit only the setup changes?

Chosen: revert. The 47 fixes are not in the repository; §6b records what they were.

7. The overvoltage cut-off in front of the sensor trackers (8 October)

§4 chose a tracking regulator per sensor group "behind a 100 V-class overvoltage cut-off", and named an LTC4368-class controller as the candidate. Its datasheet had never been retrieved. Retrieving it, and looking for an alternative, turned the candidate into a choice.

What the part is for

The three sensor regulators (TPS7B4253-Q1, one per group) survive at most **45 V** on their input. This board's battery rail reaches **73.3 V** for about 50 µs on ISO 7637-2 pulse 2a, a standard automotive surge test. So the regulators need a fast automatic switch in front of them:

- normally **on**, passing battery power through;
- above about 45 V it switches **off** within a few microseconds, and a 47 µF capacitor after it keeps the sensors powered meanwhile;
- when the surge has passed, it switches back **on**.

It is a small controller chip driving one power transistor. It must stay **on** through the 40 V load dump, which lasts 400 ms, far longer than the capacitor could bridge.

The question

Asked: which overvoltage cut-off controller should feed the three sensor trackers?

The first time it was asked, the options were given only as datasheet figures, and the owner replied that they could not decide between two part numbers. The question was then explained as above, with this

comparison:

	TPS48110-Q1 (Texas Instruments)	LTC4368 (Analog Devices)
Survives	100 V	100 V
Switches off in	2.6–4 µs after the threshold	under 8 µs
Automotive-qualified (AEC-Q100)	**yes**, Grade 1	**no**: C, I and H grades only (H = −40 to +125 °C)
Around it	a few more parts: charge-pump capacitor, unused current-sense pins tied off	simpler, 10 pins
Datasheet	SLUSEE5E (April 2026), from ti.com	Rev A, read through a mirror because analog.com timed out
Extra check needed	a 60 V/µs limit on how fast its input may rise	—

Recommended: TPS48110-Q1. Electrically, either would do the job. The deciding point is qualification. AEC-Q100 means the chip has passed the automotive industry's stress tests for temperature cycling, vibration and lifetime. Every other IC on this board has it, and the LTC4368 would have been the one exception, on the part that protects the sensor supply. The standing rule puts reliability before cost and convenience.

Chosen: TPS48110-Q1 ("ok use the recommended one").

Consequences:

- **Slew limit:** the input slew rate on pulse 2a's edge has to be checked against the 60 V/µs limit. The `sensor_rail` rewrite does that.
- **Threshold window:** the OV threshold must sit above the 40 V load dump at its lowest tolerance, and trip early enough that the regulators' input stays under 45 V. That includes the rise during the controller's 2.6–4 µs reaction time. The window is narrow, and the `sensor_rail` rewrite pins it.
- **The switch transistor** is the board's existing 100 V N-FET, IPD90N10S4-06, so there is one part number fewer to buy.

What the simulation then showed. Pulse 2a was applied through the real pin-21 filter. The trackers' input peaks at about **22 V**, nowhere near 45 V, at every corner. TI's own inrush network does most of that: the 22 nF from gate to ground holds the FET's gate, so the FET can only follow the surge up to its gate voltage. The OV comparator then opens it fully once the rail passes 40.5–42.1 V, and that timing turned out not to be critical. The 60 V/µs slew limit is met by feeding the controller's supply pins through 100 Ω and 22 nF, which gives 4.9 V/µs. The 47 µF hold-up is enough even aged to 33 µF with all three trackers at their limit. One thing it showed about the rest of the board: with the filter inductor not saturated, the clamp node reaches **76.6 V**, not the 73.3 V `transient_clamp.cir` reports. Every part there is 100 V class, and the figure is now a check.

A limit this design does not remove

The trackers' outputs survive −1 V to +45 V, so a sensor supply wire shorted to battery is survived at 24 V (jump start) and at 40 V (load dump). A short to battery **at the same time as** a pulse 2a surge (73.3 V) would exceed it. That is a double fault, and it is recorded rather than designed out.

8. The boost inductor, revisited (8 October)

§2b approved two changes to the injector boost converter: feed the controller's supply (VDD) from the clamped output side, and use a 100 µH inductor (Würth 7447709101). Building it showed the 100 µH part does not fit every rule.

What the inductor and its sense resistor have to do

The TPS40210-Q1 measures inductor current through a small sense resistor, and that one resistor sets two things:

- **Stability.** TI's Equation 9 caps the resistor at 80 % of VDD × L × fSW / (60 × (VOUT + VD − VIN)). Below it the converter is stable; above it, it goes "subharmonic" and oscillates at half its switching frequency. The worst corner is the 6 V cranking dip. Feeding VDD from the boost output raises VDD from about 5 V to about 32 V while cranking, which is the whole remedy.
- **The current limit.** The controller cuts each cycle when the resistor's voltage reaches 120–180 mV, a 1.5 : 1 spread over tolerance and temperature. The limit has to sit high enough never to trip in normal running (TI's Equation 49), and low enough that in a fault the inductor does not saturate badly.

The first question

Asked: with the approved 100 µH inductor, all three rules cannot be met strictly. Allow the fault current limit to reach the inductor's softer 30 % saturation point, or find a bigger inductor?

The owner's answer: "150 µH inductor comes in size 10x10x4 SMD package."

Checked against Würth's own datasheets:

- **7447714151, 10 × 10 × 4 mm:** 150 µH, Isat 1.4 A typ. It cannot work. Normal cranking peaks at about 1.14 A, so TI's rule puts the current limit's minimum near 1.8 A and its maximum near 2.7 A. The part would saturate in normal running, not just in a fault.
- **7447709151, 12 × 12 mm:** 150 µH, Isat 2.7 A (10 %) and 3.3 A (30 %) typ, in the **same footprint** as the approved 100 µH part.

The second question

Asked: which boost inductor?

	100 µH 7447709101 (as approved)	150 µH 7447709151
Sense resistor	50 mΩ	62 mΩ
Current limit, min–max	2.4–3.6 A	1.9–2.9 A
Limit max vs 10 % saturation	16 % over	7 % over
Limit max vs 30 % saturation	5 % under	14 % under
Stability margin (Eq. 9, 80 % rule)	1.07×	1.30×
No false trip while cranking (Eq. 49)	1.27×	1.07×

Recommended and chosen: 150 µH, Würth 7447709151, with a 62 mΩ sense resistor. It has more stability margin and less fault-current saturation, in the same footprint. This revises the 100 µH in §2b; the VDD-from-output remedy stands.

Why neither choice is strict: the 1.5 : 1 spread of the current- limit threshold means the limit's maximum is always 1.5 × its minimum. The minimum has to clear normal cranking current, so the maximum lands above the first softening point of any inductor this size. Partial saturation during a fault is not damaging: the controller still cuts the current every cycle.

How it was built

- **VDD from the output.** A follower off the 100 V rail gives VDD 34.3–45.0 V. It is 12V_GATE's circuit with a 39 V zener (BZG03C39, same Vishay family as the 36 V clamp): a PZTA42-Q pass transistor, 2 × 3.3 kΩ collector resistors, and a PMBT3904-Q limiting at 11.6 mA. A follower off a 100 V rail needs that protection whatever it feeds. If VDD shorted, the follower would otherwise burn.
- **Start-up from the battery.** The old 220 Ω feed became a 180 Ω start-up path, with its 36 V clamp, into a Schottky (SS20PH102, already on the board). The Schottky stops the follower pushing about 0.1 A back into the battery rail. 180 Ω, not 220 Ω, buys back the Schottky's drop: VDD is 4.77 V at the 6 V cranking dip, against the 4.5 V start threshold.
- **Found while building it:** the oscillator's timing resistor returns to VDD, and TI specifies the frequency −20 % / +7 % across VDD. Equation 9 scales with frequency, so the stability check has to use −20 %. At that corner the 150 µH design keeps 1.12×. The 100 µH design would have failed outright (42.9 mΩ against its 50 mΩ). The choice in this section was the right one for a reason nobody had seen yet.
- **Stated, not hidden:** the switch FET's datasheet gives only a typical gate charge. At twice that, start-up margin falls to 17 mV. Until the output passes about 36 V, the controller runs from the battery feed with a

shallow ramp, so subharmonic operation is possible for the first milliseconds of a cranking start, not in running.

---

9. The three capacitors that are not ceramic (8 October)

Every chip capacitor on the board is a multilayer ceramic, packaged automatically by `assign_passives.py`. Three are not, because each needs something a ceramic cannot give: a lot of capacitance at high voltage, or a controlled internal resistance. They were left open until now.

Ref	Job	Chosen
C313	bulk on the MCU's 3.3 V supply	KEMET T599D157M006ATE025, 150 µF polymer tantalum, **plus R306, 22 mΩ 1 %** in series
C211	holds the sensor supply up while the cut-off is open	Chemi-Con HHXC630ARA470MHA0G, 47 µF 63 V hybrid polymer, 8 × 10 mm
C401 + C420	the injector's 100 V energy tank	**two** Chemi-Con HHXC800ARA820MJC5G, 82 µF 80 V hybrid polymer, 10 × 12.5 mm, in series = 41 µF, 160 V

All three are AEC-Q200, surface-mount, and rated to 125 °C or higher. Chemi-Con figures are from its 2026 catalogue (CAT. No. E1001A); KEMET's from the part's own specification sheet.

The idea behind all three: ESR

Every real capacitor has a small resistance in series with it, its **ESR**. In wet aluminium electrolytics it rises steeply in the cold, because the liquid inside thickens: typically 4–10× at −40 °C. A **polymer** or **hybrid polymer** capacitor replaces most or all of that liquid with a conductive solid, so its ESR barely moves. Chemi-Con guarantees the HXC series' impedance at −55 °C is **at most twice** its 20 °C value. That one property is why all three choices are polymer types.

C313 — the bulk capacitor needs a *minimum* resistance

`mcu_pdn.cir` showed the bulk capacitor's ESR is not just a loss but the damping for a resonance between the regulator and the capacitor. Too little ESR and the supply rings; too much and it stops filtering. The requirement (`PDN-BULK`) was therefore a band: 20–50 mΩ.

Two findings changed how it was met:

- **A capacitor datasheet gives a maximum ESR, never a minimum.** No part can be bought "at least 20 mΩ". So the floor is a separate resistor: 22 mΩ, 1 %, in series with a capacitor whose own ESR is 25 mΩ at most. Even if the capacitor's ESR were zero, the pair is 21.8 mΩ.

- **The ceiling was conservative.** A sweep of the same model shows the 100 mΩ target holds from about 17 mΩ up to about 95 mΩ. The pair's worst high end is 72.2 mΩ: resistor +1 % plus the capacitor at KEMET's 2× end-of-life limit. Both ends pass at the smallest capacitance the datasheet allows (96 µF): 71.9 and 86.9 mΩ.

No question was put to the owner: the resistor is the only way to meet a requirement both ends of which are needed.

C211 — the sensor hold-up

When the TPS48110-Q1 opens for a voltage spike (§7), this capacitor keeps the three sensor regulators running. The only new demands are a voltage rating above the 40 V load dump the switch lets through, and a capacitance that survives aging. The HXC part gives 63 V (1.58× over 40 V). At −20 % tolerance and −30 % end of life it is 26.3 µF. `sensor_rail.cir` now runs its two worst corners at 26.3 µF, not the 33 µF it used before, and still passes: the rail's lowest point is 9.2 V. Its ESR drop at −55 °C, aged, with all three regulators at their limit, is 0.25 V. No question was needed.

C401 — the injector tank, and the question

What it does. The boost converter can only trickle-charge this capacitor. The capacitor alone delivers the injector's opening pulse: 18 A within about 40 µs, every injection. Its ESR is in series with the injector. At 18 A, every ohm of ESR costs 18 V of push and slows the opening.

What the search found. The tank sits at 100 V and reaches 104 V, so it needs a part rated above 100 V. In the catalogues searched, **no capacitor rated above 100 V publishes its ESR at all**, cold or warm. Chemi-Con's 160 V parts give only a 120 Hz loss figure. Vishay's specified-cold series stops at 80 V. Parts rated up to 80 V do publish it, including HXC.

Asked: "C401 is the injector's 100 V energy tank … No single capacitor rated above 100 V publishes its cold ESR. Which way?"

	2 × 80 V hybrid in series (recommended)	1 × 160 V wet electrolytic
Part	2 × Chemi-Con HXC 82 µF 80 V, SMD	1 × Chemi-Con GXF 51 µF 160 V, radial
ESR at −55 °C, aged	224 mΩ, from the datasheet	not specified; estimated ~8 Ω
Time to 18 A at −40 °C, lowest rail	45 µs (38 µs nominal)	never reaches 18 A at the estimate: 10.2 A at most
Cost and space	one extra can, 10 × 12.5 mm	cheapest, one part

The owner's answer: "2× 80 V in series (Recommended)."

Why it matters: the injector controller ends the opening phase when the current reaches its peak threshold. A tank that cannot push 18 A through its own resistance never ends that phase, and the engine misfires. That would happen on a cold start, late in the part's life, which is the hardest fault to find in the field.

How two in series work:

- **Capacitance halves, voltage rating doubles.** 41 µF, 160 V. The sims now run at 41 µF. The droop per injection is 8.3 V (it was 7.3 V). On the smallest stack the datasheet allows (−20 %, then −30 % aged) it is 14.9 V, so the rail never falls below 85.1 V. Every check that used the old 92.7 V low point now uses that figure. The gate-driver bank node rises to 104.1 V, since each hold-cutoff bump is 2.45 V on 41 µF. Recovery time does not change; it depends on charge, not capacitance.
- **The bleed balances the stack.** Two capacitors in series only share the voltage evenly if something forces them to. The 22 kΩ bleed that `BOOST-BLEED` already required is now 2 × 11 kΩ, one across each can (R401, R435). With the worst leakage mismatch the datasheet allows, each can settles at 52.4 V on an 80 V part. The first charge from 0 V is the worst moment: it splits by capacitance alone, so 71.0 V can land on one can. That is 1.13× under its rating. Total bleed current and power are unchanged.
- **Ripple:** the injection pulses plus the charger's switching current, counted as if it never stopped, are 1.06 A rms against 2.1 A rated at 125 °C.

All of it is executable arithmetic in `run_sim.py` under `boost_converter`, beside the converter's own datasheet checks.

---

10. A filter on each new supply feed (8 October)

POWER-PATH (§2a, §5) moved the injector and EGR currents onto connector pins 04 and 06. That was right for current and heat, but it took the board's two noisiest loads away from pin 21's EMI filter. Each new feed had only its reverse FET and 4.4–8.8 µF of ceramic capacitors. The question was whether that is enough to meet the radio-interference limit this project designs to. The limit is CISPR 25 Class 5, the strictest automotive class (research memo 06), the same target pin 21's filter is held to.

What makes the noise

Both loads switch large currents on and off many thousands of times a second, straight from the battery wire:

- **Injector hold.** After the 100 V opening pulse, each injector is held open at about 10 A by switching the battery on and off ("chopping").
- **EGR valve motor.** Driven by PWM: full battery on and off, up to 6.4 A at stall.
- **Boost converter input.** A smaller, smoother 150 kHz ripple.

Each on-off edge sends a spray of harmonics down the harness. A radio receiver near the wiring would hear it, and the standard limits how much is allowed.

What the model found

New block `feed_emi.cir` reproduces CISPR 25's measurement: the standard 5 µH / 50 Ω test network (LISN), 200 mm of harness, then the board. `run_sim.py` adds up each load's harmonics against the Class 5 limits, band by band, with the peak and average detectors.

	Worst point against Class 5
VBAT_INJ as drawn	**33 dB over**
VBAT_EGR as drawn	**48 dB over**
VBAT_INJ with the filter	17.7 dB under (12.2 dB under with aged capacitors)
VBAT_EGR with the filter	9.3 dB under

48 dB is about 250 times too much voltage. No layout trick closes a gap that size; it needs a filter.

The filter, per feed

- **Coilcraft XAL1510-103**, 10 µH, rated 16 A for a 20 °C rise. Between the reverse FET and the loads.
- **Two 82 µF / 80 V hybrid capacitors** (the same Chemi-Con HXC part as C401) on the load side. These supply the chopping current, so the inductor only carries the average.
- **Two dampers:** another HXC each, in series with a 0.68 Ω resistor.
- The existing 2 × 2.2 µF ceramics now sit on the connector side, which makes it a two-capacitor filter.

Both feeds get the identical circuit: one parts list, one layout block.

Why the dampers. An inductor and capacitor together form a resonant circuit, here near 3 kHz. Two of the loads on the injector feed, the boost converter and the hold-current loop, try to draw *constant power*. To the filter that looks like a negative resistance: about 0.94 Ω at a 9 V cranking dip with a hot injector coil. If the filter's own impedance peak is higher than that, the supply can oscillate. Undamped it peaks at 1.24 Ω. With the dampers it peaks at 0.64 Ω, even with end-of-life capacitors, a

margin of 1.47×. The damping is a plain resistor, not a wet capacitor's internal resistance, because that rises about 8× at −40 °C (§9), exactly when cranking makes it matter.

Why this inductor. The Würth 10 µH alternative saturates at 8.8 A typical and is rated only to 75 °C ambient. The Coilcraft part saturates at 26 A and is rated to 165 °C. Its one limit is 55 V across the winding. The model puts ISO pulse 2a through the filter: at most 48.7 V across it, 1.12× inside. In return the filter swallows most of pulse 2a, so the feed's loads see at most 28.7 V, where VBAT_PROT sees 73.3 V.

A requirement this puts on the firmware

The hold chopping and the EGR PWM frequencies have not been chosen; they are firmware. Harmonic strength in the radio bands grows with switching frequency, so the filter was sized at **25 kHz**. At 50 kHz the EGR feed would be 1 dB over. **Both must stay at or below 25 kHz.** That is above hearing, which is the usual reason for choosing about 20 kHz anyway.

What it costs, and what would change it

Two inductors (15 × 16 mm), eight hybrid capacitors (10 × 12.5 mm) and four resistors: 14 parts, roughly US$15–20 a board. No question was put to the owner. Without the filter the board fails Class 5 by 33–48 dB, and the only alternative is a looser emissions class. That is the owner's call. A stationary genset has no car radio, so a lower class may be defensible, but it is not this log's decision to make. If the owner relaxes the class, these filters can shrink.

Decided 12 October 2026: the owner keeps CISPR 25 Class 5. The filters stay as designed. The PCB layout is held to Class 5 practice too: tight switching loops, an unbroken ground plane, and filters placed at the connector. The layout guide carries this.

Limits of the model. Like `emi_filter.cir`, it covers differential mode only. Common-mode noise, both wires moving together against the chassis, needs a layout to model. The 100 ns switching edges and the in-phase sum of harmonics are deliberate worst cases.

---

11. Firmware on the PC first (10–11 October)

The hardware was complete to schematic, but no firmware existed. A competing team already runs firmware on their board, with an Arduino faking the engine signals. The question was how to catch up without waiting for our own PCB.

The choice

The control software is written as **plain C that touches no hardware** (`firmware/core/`). It runs on the PC against a **simulated engine** (`firmware/sim/engine.py`). The same C files later run unchanged on an NXP S32K148 evaluation board, and after that on our own board. Every control and safety function is proven first on the PC, where a mistake costs nothing.

All the firmware sees is one table of numbers (`ecu_io` in `ecu.h`). The inputs are sensor voltages and switch states. The outputs are injection commands, the metering-valve duty, the watchdog pin and the CAN messages. The firmware has four entry points: start up, a crank tooth arrived, a cam pulse arrived, and one millisecond passed. The simulator fills the table and reads it back, and the S32K148 port will do the same from its pins. Injection is given as **"start at time T, for D microseconds"** per cylinder, which is what the chip's timer executes. That keeps the simulator's time step coarse without making the fuel amounts coarse.

What the firmware does

- **Reads the crank wheel.** A 60-2 wheel has 58 teeth and a gap where two are missing. The gap shows up as a tooth period more than 1.5× the one before. Two gaps 58 teeth apart mean "synchronised". The cam pulse, once per two revolutions, says which revolution has cylinder 1 firing.
- **Injects** once per cylinder per cycle, 5° before top dead centre. It schedules at least 1° ahead, so the start time is never already past.
- **Governs speed.** It holds 1500 rpm, or falls 4 % from no load to full load when the droop switch is closed. On reaching 400 rpm the setpoint **ramps up at 500 rpm/s** rather than jumping, explained below.
- **Holds rail pressure** at 1000 bar by driving the metering valve, treated as normally closed: more duty, more fuel flow. See the final review below.
- **Checks every sensor.** One that stays out of range for 100 ms raises a fault code. From the first moment it goes out of range, its last good value is held.
- **Shuts down and latches.** Above 1650 rpm (checked on every tooth), on loss of the crank signal, or on ignition off. It stays shut down until the ignition is cycled. The trip cancels any injection not yet started.
- **Talks J1939 to the DSE4522:**
- engine speed every 20 ms
- coolant temperature every 1 s
- oil pressure every 500 ms
- active fault codes every 1 s

What the simulator proved

Eleven scenarios, run with one command (`.venv/bin/python firmware/sim/run_sil.py`), all pass in 36 s:

#	Scenario	Result
1	Start	synchronised in 116 teeth; 1500 rpm by 5 s; peak 1534 rpm
2	0 → 20 kW load step	dips to 1449 rpm, back within ±15 rpm
3	20 → 0 kW load rejection	peaks at 1547 rpm, well under the 1650 trip
4	Droop switch	speed falls 3.8 % (want 4 ± 1 %)
5	Runaway (stuck injector)	trips at 1650.6 rpm; no new injection after
6	Crank signal lost	shuts down 50 ms later, fault SPN 636
7	Ignition off	shuts down with no fault code
8	Coolant sensor unplugged	fault SPN 110 FMI 3 after 1.0 s; engine keeps running
9	CAN timing and content	every message on time to the ms; speed within 0.1 %
10	Rail pressure	within 1.6 % of 1000 bar at full load
11	Watchdog	WDI falls every 10 ms

There are also 46 unit tests on the separate pieces (`make -C firmware/sim test`).

What an expert review caught

Halfway through, a separate reviewer acting as a diesel-ECU firmware specialist read the code. Every finding was reproduced as a failing test before it was fixed. The ones worth knowing:

- **The start nearly tripped overspeed.** On reaching 400 rpm the speed loop asked for 1500 at once, fuel hit its limit, and the engine peaked at 1647 rpm, 3 rpm short of the trip. The ramping setpoint brought the peak down to 1534.
- **A crank-sensor noise spike read as overspeed.** One extra edge in mid-tooth made the speed read 1700 rpm. An edge arriving in less than half a tooth's time is now ignored, because no engine accelerates that fast.
- **A cancelled injection could still fire.** The cancel did not change the sequence number a timer driver watches. Now it does.
- **A hot engine looked like a shorted sensor.** With the placeholder coolant sensor, 0.2 V is a real 129 °C. That would have been flagged as a short and replaced by a harmless 90 °C. The coolant short threshold is now 0.1 V.
- **A stray cam pulse shifted everything by a revolution.** The engine would have injected on the wrong stroke. A cam pulse in the wrong revolution now stops injection until the next one agrees.
- Smaller fixes:
- a timestamp 1 µs out of order read as signal loss

- an out-of-range sensor's raw value was used during the 100 ms debounce

What the final review caught

After all ten steps, a second reviewer read the whole branch. Again, each finding became a failing test before it was fixed.

- **The metering valve's polarity contradicted the safety design.** This firmware spec had assumed a normally-open valve. The parent spec's trip module stops fuel by *cutting the valve's supply*, and that only works if the valve is **normally closed**: unpowered, no fuel. With a normally-open valve, cutting its supply opens it fully. The runaway run showed the rail climbing to 1695 bar after the trip. The firmware now assumes normally closed, to agree with the parent spec. Polarity is a single constant (`METER_NC`), and shutdown always commands the closed position. **The real valve still has to be identified.** If it is normally open, the constant flips *and* the trip module's contacts must move to the injector supply.
- **A stray edge before cranking blinded the decoder for 30 s.** The noise filter compared every tooth with a stale 30-second "period". After any pause longer than 50 ms, the decoder now starts afresh.
- **A 1 ms ignition flicker cleared a latched trip** while the engine was still turning, and the ECU restarted it. The latch now clears only after the ignition has been off for 500 ms with the engine stopped.
- **The speed reading could drop to 0 for one tick** if a tooth arrived just after the tick read the clock. That doubled the fuel demand for a moment. The time comparison is now signed, as it already was in the decoder.
- **Scenario 5 could not tell an edge-checked trip from a late one.** With the per-edge overspeed check deleted, it still passed. It now checks that the ECU is shut down on the very edge it first reads above 1650 rpm, and that rail pressure falls after the trip. With the check deleted, the scenario now fails.

Choices made while building

- **Signal-loss timeout of 50 ms, not 100.** The scenario requires shutdown within 100 ms. At 400 rpm the longest normal gap between edges is 7.5 ms.
- **Watchdog kicked every 10 ms.** This settles the open item on the MCU sheet. The TPS3850's window (its SET0/SET1 straps and CWD capacitor) has to accept a 5–20 ms kick period. That is well inside the 26.67 ms cylinder interval the spec requires.
- **Tuned gains.** These are placeholders until real engine tests:
- speed: 0.5 mg/rpm proportional, 2.0 mg/(rpm·s) integral
- rail: 0.002 duty/bar proportional, 0.01 duty/(bar·s) integral

The gentler of the two passing sets was kept. A real engine has delays this model lacks. - **A stray cam pulse or a noise edge** does not stop the engine. A single spike should not shut down a genset.

What is assumed, and how to change it

Each assumption is a single named constant in a header, marked `assumed` or `calibration`: - the 60-2 wheel and cylinder 1 TDC at 90° after the gap (`CRANK_TDC1_DEG`) - firing order 1-2-3 - start of injection, 5° before TDC - the injector flow law - fuel at full load, 34 mg (sets the droop) - every sensor's volts-to-units table (`sensors.c`)

The engine model's constants are guesses for a 3-cylinder genset diesel, also marked. When the real engine's figures arrive, each is a one-line change, and the scenarios say at once whether the control still holds.

Out of scope, and next

Not done yet: - injector current regulation (that lives in the driver chip) - pilot and post injections - real calibration maps - EGR control

The next step is the S32K148 evaluation board with an Arduino generating the engine signals: the competitor's setup, but with control logic that is already proven.

---

12. Logs a production ECU keeps (11 October)

This ECU goes into a production genset that runs for months at a time. Until now, everything the firmware knew disappeared at power-off. A technician arriving at a stopped set would have found nothing.

Four records

All four live in one **persistent image** of about 3 KB. That fits the S32K148's 4 KB emulated EEPROM.

- **Fault memory.** Up to 16 faults, kept after they clear. Each has an occurrence count, the first and last time it was seen, and a **snapshot taken the first time**:
- speed
- load
- fuel
- rail pressure
- coolant
- oil
- battery
- state

It answers "what was the engine doing when this happened?" - **Event log.** The last 64 of: - power-ups and their cause (a watchdog reset is a firmware hang caught in the field) - ignition on and off - crank, run and shutdown, with the reason - faults set and cleared - stored-fault clears - **Counters.** - ECU powered time and engine hours - starts - trips by cause - watchdog resets - time at each quarter of rated load - **Trip recorder.** For the last two overspeed or crank-signal trips: speed, fuel, rail pressure and state every 20 ms, from 2 s before to 1 s after. Normal ignition-off stops are not recorded, or the daily stop would overwrite the last real trip.

Surviving power loss and flash wear

- **Two copies.** The host keeps two copies of the image and writes them alternately. Each copy carries a sequence number and a CRC-32, and at power-up the newest good copy wins. A power cut in the middle of a write loses that one save, never the history.
- **Sealed copy for the host.** The image the host writes is a sealed copy, so the 1 ms tick can keep updating the live one during the write.
- **Saves only from the main loop.** A save happens on:
- a fault's first occurrence
- every trip (1 s after it, once the recording is complete)
- entering shutdown
- each start
- every hour of running

The rate is capped at one save per 10 s, except a trip. - **Wear.** That is about 30 saves a day, roughly 110 000 per copy over 20 years. The S32K148's storage has to be chosen for that, in the dev-kit spec.

Reading it

Over J1939, which the DSE4522 and standard service tools understand: - **stored faults (DM2)** - **clear stored faults (DM3)** - **fault snapshots (DM4)** - **engine hours**: on request, and broadcast every 10 s until we know whether the DSE asks for them (U4)

The event log and trip recorder need a laptop service tool over CAN (UDS), which belongs in the dev-kit spec. Until then, the simulator reads them straight from the image.

The watchdog needs the main loop now

Until now the 1 ms tick pulsed the watchdog on every pass. A main loop hung while interrupts still ran would have kept the watchdog happy forever, which is exactly the failure a window watchdog exists to catch.

The tick now pulses only if the main loop has checked in within 50 ms. If the loop hangs, the pulses stop, the TPS3850 resets the processor, and the next power-up logs a watchdog reset.

Three more fixes for long running

- **The millisecond counter wraps every hour by design.** It used to wrap after 49.7 days, which would have broken message timing once in the field.
- **A lost crank tooth is logged as a crank-sensor fault.** It used to be logged as overspeed, because the missing tooth makes that edge's speed read high. The decoder also drops its speed history on any fault, so one bad edge can't raise a second, false overspeed.
- **A fault message too long for one CAN frame is sent whole or not at all.** It is never half-sent.

What the tests caught while building

- **Faults read "active" after power-up.** A fault saved while active still read "active" after a power cycle. Load now marks every stored fault as previously active, because nothing is active until it is seen again.
- **Scenario 11 couldn't see the watchdog stop.** With the watchdog newly tied to the main loop, the simulator, which didn't run the main loop yet, stopped pulsing after 50 ms, and scenario 11 still passed. It now also checks that the pulses continue to the end.

What the final review caught

A fresh reviewer, acting as an off-highway ECU firmware specialist, found no critical defects. It did find four that would hurt in months of field running. Each now has a test that failed first: - **A loose sensor connector flooded the event log.** Every flicker logged "fault set" and "fault cleared", and in 10 s that pushed the overspeed record out. Each fault's set and clear is now logged once per ignition cycle, and ignition-off only after 500 ms off, so a bouncing contact logs nothing. - **A service tool reading fault snapshots could silence the DSE's fault list.** Long multi-frame messages queue one at a time, and the tool's requests always won. The active fault list (DM1) now goes first. - **A watchdog reset was never saved on its own.** If the operator power-cycled before anything else triggered a save, the evidence of the hang was lost. A watchdog-reset boot now requests a save, still under the 10 s limit, so a boot loop can't wear the flash. - **A packet of a long message could be dropped** if the CAN queue was full at that moment. It is now retried 1 ms later.

Proof

- **78 unit tests.**
- **15 scenarios.** The four new ones:

#	Scenario	Result
12	Runaway, then power cycle	overspeed fault stored with its snapshot (1650 rpm), shutdown event logged, trip recording kept (last sample before the trip 1633 rpm, 50 after)
13	Power cut halfway through a save	reload takes the previous save; nothing else lost
14	Stored faults over CAN	DM2 returns the stored fault; DM3 clears it and leaves an active one alone
15	One lost crank tooth at 1500 rpm	shutdown, stored as crank sensor (636), not overspeed

Still open (dev-kit spec)

- **Storage choice.** Pick storage for about 110 000 writes per copy.
- **Laptop service tool.** It reads the event log and trip recorder over UDS.
- **DM3 acknowledgement.** Whether the DSE expects one (U4).
- **Interrupt masking.** Mask the tick during the sealed-copy step on the real chip.

Addendum, 14 October: the Protect lamp

The problem: the DSE4522's configuration (`field-evidence-2026-09-18.md`) treats the DM1 **red stop** lamp as a *warning* and shuts down only on the **ECU Protect** lamp. Our firmware set red on its shutdown faults but never Protect. So after our ECU cut the fuel for overspeed, the DSE would have seen only the speed falling and shut down on *under-speed* (1310 rpm), logging the wrong reason.

Chosen: a shutdown fault (overspeed, SPN 190; crank signal lost, SPN 636) now sets **red and Protect** (DM1 byte 1 = 0x11). A sensor fault sets amber only (0x04), so a bad sensor never stops the set through the DSE.

Proof: three unit tests (79 in all): overspeed and crank loss each send 0x11; an open coolant sensor sends 0x04. All 15 scenarios still pass.

Still to confirm on the machine: that the DSE logs "ECU Protect" rather than under-speed when our ECU trips. That's a commissioning check.

---

13. The board, prepared for hand routing (11–14 October)

The question: should the PCB be laid out here, or by a person? And since the new ECU must fit somewhere on the engine, where does it go?

Chosen (owner, 11 October): - **I prepare, a person routes.** The board file arrives with every part placed by block and every rule set. A layout engineer routes it by hand in KiCad 10. - **The board goes in the old ECU's die-cast housing.** The housing is also the heat sink. So the outline, the screw holes, the cooling contact areas and the J1001 header all copy the old unit.

What exists now

Item	Where
The board, 379 parts, 373 nets, placed by block	`hw/ecu25kva.kicad_pcb` (KiCad 10)
Stack-up, computed widths, net classes, housing placeholders	`hw/pcb_rules.py` (tested by `hw/test_pcb_rules.py`)
The generator	`hw/gen_pcb.py`. On a routed board it updates only the rules
The check	`hw/check_pcb.py`. Footprints, nets, classes, widths, DRC: all pass, 499 connections unrouted (expected)
Guide for the person routing	`docs/layout-guide.md` (the bible's PCB Layout Guide)
What to measure on the old ECU	inspection brief, Group 6

The decisions inside it

Item	Choice	Why
Stack-up	4 layers, 1.6 mm, Tg ≥ 170 °C; 2 oz outer, 1 oz inner; L2 a solid ground plane	Engine bay; high current on the outer layers; the plane is never cut
Track widths	Computed from current by IPC-2221, for a 20 °C rise. Never typed	The spec named IPC-2152; IPC-2221 has a formula and comes out wider, so it errs safe. 18 A → 5.35 mm, 6.4 A → 1.3 mm on 2 oz
Clearances	IPC-2221B, external, uncoated: 0.6 mm for anything from 31 V up (boost voltage; the EGR gates, which reach 52 V on a load dump); 0.2 mm for CAN and analogue; 0.15 mm for logic and low-voltage power on fine-pitch pins. Battery power at 0.3 mm is a **judgement**, not a table row: the table gives 0.1 mm up to 30 V	Safe even before conformal coating
Net classes	Ten: PWR_HI, RTN, PWR_LOOM, PWR, PWR_LV, HV100, HV100_SIG, CAN, ANALOG, Default	See below for the ones that differ from the plan
The EGR feed (pin 6)	Sized for its 10 A loom fuse, not the 6.4 A load: 135 % of rating (a blade fuse may carry that for up to 30 minutes, ISO 8820-3) = 13.5 A → 3.6 mm	Nothing on the board limits that feed below the loom fuse
Unclassified power nets	The check fails if any net touching a power part (power FET, power diode, inductor, fuse, milliohm shunt) has no class	A new power net can't silently get a thin trace
Pads inside one part	Exempt from class clearance (a custom rule)	A 0.5 mm-pitch chip can't meet a 0.3 mm class rule; that spacing is the maker's. The classes govern the copper the router adds
Single ground join	All high-current returns meet the board ground at J1001's battery-ground pins, and only there	Keeps injector and switching current out of the sensor ground (layout guide §4)

Three changes from the plan, found while doing it: - The injector's 100 V nets carry 18 A, not 1 A. The plan sized the 100 V class for 1 A. But the injector's peak current flows through the boost rail, the bank nodes and the low-side drains. So that class takes the 18 A width. A new class, **HV100_SIG**, gives the small nets that sit at boost voltage (gates,

bootstraps, divider midpoints) the 0.6 mm clearance without a 5 mm default width. - **Ground has its own class (RTN).** Ground was planned under the 18 A class, with its 0.6 mm clearance. That put 0.6 mm between every ground pin and its neighbour, which no fine-pitch chip meets. Ground, with the injector shunt tops, keeps the 18 A width at the default clearance. Anything at 100 V still gets 0.6 mm from its own class. The same reasoning gives 3V3, 5 V and the boost controller's pins the PWR_LV class at 0.15 mm. - **The fused pin-21 path is 6.4 A class, not 18 A.** It sits behind the 5 A fuse F101. The injector's power comes in on pin 4.

Placeholders, and what the measurements change

Item	Placeholder	Settled by
Board outline	180 × 130 mm	Group 6.1–6.2
Mounting holes	4 × M3, corners	6.1–6.2
Housing contact band	top 25 mm	6.1–6.2
J1001 footprint	6 large + 4 × 22 at an assumed 2.5 mm pitch, through-hole, marked "DO NOT FABRICATE"	**6.3**
Battery-ground pins	1 and 2, assumed. The schematic leaves them unconnected	5.1, 6.4
D806 footprint	SMA	the trip-loop pin, 5.2

When the measurements arrive, `HOUSING` and `REGIONS` in `pcb_rules.py` change. Before routing, re-run the generator. After routing starts, move the outline and J1001 by hand.

One finding the owner should know. The spec's placeholder outline was 160 × 120 mm, and **the parts don't fit on one side of it**. Packed, they need about 15,300 mm^2. At 160 × 120, the power-input block alone had no room beside the connector. The placeholder is therefore 180 × 130. If the old housing's board is smaller, our parts go on **both sides**. That is normal for an ECU, but it needs the housing's inside depth on both faces (Group 6.1 asks for it).

Two things the schematic doesn't have yet, which the layout guide reserves room for: - a common-mode choke at the power input and on each CAN pair (spec §6, needed for CISPR 25 Class 5) - test points

One KiCad trap. kicad-cli 10 silently ignores a custom-rules file it can't parse; KiCad 10 needs units on every number (`0mm`, not `0`). So `check_pcb.py` confirms the rules file is the generated one before it trusts the DRC.

---

14. Calibrate from the old ECU's own recording (14 October)

The evidence: a 35-minute XCP recording of the original Bosch ECU on a running 3-cylinder engine at 1500 rpm (`refs/field-evidence-2026-10-14.md`).

Asked: should the firmware take its numbers from it? **Owner:** yes, "we need it to work right."

What changed:

Item	Before	Now	From
Metering valve	normally closed (`METER_NC 1`)	**normally open** (`METER_NC 0`): full duty stops flow	current falls as pressure rises; it rises to stop the engine
Injector law	300 µs + q / (0.0009 √p)	**323 µs + q / (0.00191 √p)**	fitted to 20,473 samples, within ±6 µs
Start of injection	5° BTDC fixed	**table on fuel**: 11.5° at 6.5 mg → 15.9° at 17 mg → 14.2° from 26.5 mg	the old ECU's own values, median per mg
Rail pressure	1000 bar fixed	**table on fuel**: 827 → 1171 bar	the same
Droop	4 % only with the droop switch closed	unchanged	the old ECU runs isochronous, and so does ours with the switch open

What it costs, and what is still open: - **The trip module must move.** With a normally-open valve, cutting the valve's supply gives *full* flow, so the trip removes no fuel. The trip has to cut the injector and boost supply instead. This is an **external wiring change, not the PCB**: the trip goes in series with the injector feed, pin 04. It is written up in spec §3 item 6 (14 Oct), with the field survey in inspection brief Group 7. If the module's own contact can't carry the 10 A feed, it switches a run relay that is energised to run. Not yet wired. - **The valve's part number** is the last confirmation. If it turned out normally closed, `METER_NC` goes back to 1; both polarities are tested. - **The tables cover 6.5–26.5 mg at 1500 rpm.** Above that, and at other speeds (cranking, the run-up), they hold their end values. Those parts need the real engine. - **The old ECU dips 9.8 % on a load step,** at G2's limit. Our model shows 3.4 % on a bigger step, so it is probably optimistic. Commissioning has to measure it.

Proof: 80 unit tests, and 15/15 scenarios. The engine model now uses the measured injector and valve too. Scenario 10 checks rail pressure against the firmware's live setpoint (worst 2.3 % at no load, 1.3 % at full load).

---

15. A prototype board, buildable now (15–17 October)

The question: the production board waits on the Bosch connector and the old ECU's housing. Nothing has shown the design working. Can a board that sets both aside be built now?

Chosen (owner, 15 October): yes. Everything for it is in `prototype/` (spec `prototype/docs/spec.md`, plan `prototype/docs/plan.md`, bring-up guide `prototype/docs/bring-up.md`).

Question	Owner's answer
What goes on it	**The full production circuit.** Its eight sheets are built by the production generators, unchanged
MCU	**STM32G474VET6**, cheaper and easier to get than the S32K148. The production MCU choice is revisited once the prototype works
Dev board or one board	**One board,** MCU soldered on. A WeAct STM32G474 module is bought separately, for early firmware only
Assembly	**Fab-assembled.** The files are fab-neutral; the fab is chosen at quoting
Routing	**Done here,** by script. The owner does not route
Connector and housing	**Set aside.** Pluggable 5.08 mm screw terminals, labelled with the production pin numbers
High-current copper (16 Oct)	**Pours carry the current.** 585 of the 640 power pads are narrower than their class width, so no track can meet the class at the pad. Each power net's power parts are joined by class-width pours; tracks to small parts on a power net may be pad-width
Board size (16 Oct)	**300 × 200 mm** ("you can use bigger board dimensions"). 220 × 160 and 240 × 180 were too congested to route the injector stage at full width
Pin necks (16 Oct)	**0.15 mm, inside a part's courtyard only.** 0.2 mm (the 2 oz minimum) does not clear between 0.5 mm-pitch pins; 0.2 mm everywhere else
Layers (16 Oct)	**Stay at 4.** L2 stays a solid ground plane, never cut; signals use F.Cu, In2 and B.Cu

What it adds to production, and only to the prototype

- **Test points** on every rail, the boost rail (marked HV), CAN, the crank and cam edges and the four current senses: TP1–TP24.
- **A boost-rail monitor** `BOOST_MON`: 2 × 499 k + 24.9 k, so 116 V reads 2.82 V.
- **A battery monitor** `VBAT_MON` on the MCU sheet: 120 k / 10 k.
- **A 2 × 10 bridge header** for the future simulator board.
- **USB-C, data only.** It does not power the board, so the MCU rail has one source.

How the board is routed

- `prototype/power_router.py` routes the 29 power nets as class-width bars. It is a maze router on a 0.5 mm grid over B.Cu, F.Cu and In2. Nets negotiate for space (the PathFinder method) until no two overlap. Where a bar changes layer it gets a via array at 2 A per via. On In2, which is 1 oz copper, bars are twice as wide.

- **Freerouting 2.4.1** routes everything else. **It treats a KiCad pour as no obstacle** (in its own code, a DSN plane loads with `isObstacle=false`). So a first run routed signals straight across the power bars, which the refill would have cut. In its input each bar is now a fixed wire and each pour via a fixed via; they come out again after import. `test_router_sees_pours` guards this.
- **Ground reaches L2 through its own vias** (`gnd_fanout`), one per ground pad, so Freerouting never sees L2. It routes three layers; its vias come back through all four.
- **Freerouting leaves about 45 connections open** (run 12, 29 passes, 1 h 50 min). `prototype/finish_router.py` routes them, one at a time, with KiCad's DRC as the judge: anything DRC rejects comes out whole. What it needed, each found on a connection that would not route:
- **A start anywhere on the end's copper island**, not only its centre point. Off a 0.5 mm-pitch pin's centre is the only legal start beside a 0.3 mm clearance net.
- **Vias through a pour's interior**, with 1 mm of pour kept round the hole. A deeper bite into a pour's edge is allowed but priced as a 20 mm detour, and `necks()` judges it afterwards.
- **A last pass 50 mm round the ends on a 0.1 mm grid**, for HSB_BAT_GATE: 115 mm across the injector pours.
- **Rip-up**: other nets' tracks round the open ends, then along the line between them, are taken out; the open connection routes first, the rest after. Rounds that leave more open are undone.
- `tidy` removes the vias and stubs rip-up leaves dangling.
- **Result (17 October): nothing unconnected, 0 DRC errors, schematic parity clean** but for D806's placeholder footprint. Freerouting started from 625 nets.
- **Proofs:**
- `test_power_pours_connect`: with every track removed, every power part's pads are still joined, so no track carries load current.
- `gen_board.necks()`: every bar keeps at least 3 of 5 sample points across its width filled, outside its own pads.

Rules the routing needed (17 October)

Rule	Why	Cost if wrong
Pin escape: inside a fine-pitch chip's courtyard, a PWR or PWR_LOOM track keeps the chip's own 0.2 mm pin gap to that chip's pads, not 0.3 mm. The 100 V classes and PWR_HI keep theirs	U203 pin 13 (VSNS) lies between OVC_BST and OVC_GATE on a 0.5 mm pitch: even a 0.15 mm track dead centre is 0.275 mm from them. The chip itself holds those voltages across 0.2 mm. **Production has the same pin**	A creepage review of the 7 chips (U101, U202, U203, U301, U302, U401, U701)
Fine-pitch chips keep 1 mm clear round their courtyard in placement	Run 9 left U401, U203 and U504 walled in by their own satellites	A placement rerun

Rule	Why	Cost if wrong
R503 is turned 180° after packing	U504's gate-source resistor faced gate-down, source-up, against the driver's pin order: EGR_AH_G and EGR_HIGH_59 had to cross at 0.6 mm clearance, and EGR_A_VB was walled in	None: the same box
Finishing tracks on power nets are 0.2 mm	They reach small parts; the current rides the pours	None measurable
A power net with no pour carries its own current on tracks (`TRACK_AMPS` : 1 A on the 5 V buck's switch node and output, 0.05–0.3 A on the rest). Each track is IPC-2221 wide for that current on its layer. A net that would need more than 0.2 mm on In2 stays on the outer layers	Eight power-class nets have at most one power part, so the pour router gives them no bars. The 5 V buck's output ran 0.2 mm on 1 oz In2: 65 °C rise at the buck's 1 A, which a chafed sensor lead can hold (each sensor tracker limits at 301–520 mA). `test_track_power_widths`	A rerun if a load grows: the table is the budget

Found on the way

- **A production defect, fixed in `hw/` .** In the supervisor, the `~RESET` and `~WDO` pull-ups overlapped on the schematic. That shorted the TPS3850's reset and watchdog-fault outputs into one net. Fixed in commit 491f37e. `check_netlist.MUST_SEPARATE` now guards it.
- **That fix broke the watchdog kill (found in review, 19 October).** The TPS3850 asserts `WDO` on a watchdog timeout and never `RESET` (SBVS301B §7.3.6.3). Once the short was gone, a hung MCU no longer reset or killed the gates. **D302, a BAT54Q-7-F Schottky** (anode `~RESET` , cathode `WDT_FAULT`), puts the path back: `WDO` low pulls `RESET` to about 0.3 V. That is below Q301's 1.1 V minimum threshold, so `GATE_KILL` rises. `RESET` low leaves `WDO` alone. In production and on the prototype; `check_netlist` WDO_KILL guards it. **The cost:** the MCU is held in reset for as long as `WDO` is low, so `WDT_FAULT` no longer tells firmware which trip it was. No firmware reads it.
- **U606's library footprint** (Freescale 98ARH99066A) has a malformed courtyard. The prototype redraws it as a rectangle. **Production uses the same footprint** and passes the design-rule check only because of where it sits. Check it before production layout.
- **ST's datasheet DS12288 could not be downloaded** (st.com refused). The power-distribution check keeps production's 0.5 A load step: 70.3 mΩ cold and 78.5 mΩ warm against the 100 mΩ target. The STM32's real step is smaller.

The cost to know before quoting

- **The stack-up is production's:** 4 layers, 2 oz outer copper, 1 oz inner. The pour widths are computed for 2 oz.
- **A 1 oz quote is not a drop-in.** The bars would carry about half the current.
- **300 × 200 mm with 2 oz outer copper** is outside most fabs' cheapest prototype tier. Expect to pay for size and copper weight, not part count.

- **95 pin necks are 0.15 mm on 2 oz outer copper.** Some fabs' 2 oz minimum is 0.15–0.2 mm: ask at quoting.
- **20 small islands of pour fill** are left floating (current-sense pours on In2, three on the ground plane). They carry nothing; a fab may ask about them.

16. A service link: bootloader, end-of-line record, underspeed warning (17–20 October)

Prompted by a friend's screenshot of a comparable production ECU's firmware, a TargetLink model. It has a bootloader, an end-of-line write and an underspeed fix. Ours had none of the three.

The owner's answers (17 October)

Question	Answer
Underspeed action	**Warn; the DSE decides.** The DSE has its own under-frequency shutdown; the ECU does not stop the engine for underspeed
Link	**Both.** USB on the prototype for quick iterations; CAN channel 2 in production
End-of-line scope	**The unit record**, locked once written. A factory output-test mode is later work
Protection	**Seed-key**, and the engine must be stopped

What was built

All portable C, tested on the PC (firmware guide §16). Spec `docs/superpowers/specs/2026-10-17-service-link-design.md`.

Part	Decision	Why
Protocol	A subset of **UDS (ISO 14229)**, the same on both links	A technician's UDS tester can read the IDs; `python-udsoncan` could replace our tool
Underspeed	Below 1350 rpm for 3 s: SPN 190 FMI 18, **amber lamp only**; above 1400 rpm for 1 s clears it	The DSE shuts down on the Protect lamp, which every SPN 190 used to light. A warning must not stop the set
End-of-line record	84 bytes, two copies apart from the logs, trims applied to every input; none good means SPN 628 FMI 13	A unit that skipped the line says so on the DSE's screen
Bootloader	Image header written **last**, magic last of all; never touches its own 32 KiB	A power cut anywhere leaves no valid header: the update is just run again. Tested by cutting the power at every write
Hardware check	The bootloader runs an image only if the record's hardware revision is in the image's range	A good image for another board revision stays in the bootloader
Engine stopped	OFF, WAIT or SHUTDOWN and no tooth for 1 s; an open session ends at the first tooth; an ECU reset is refused while it turns	A laptop left connected, or a generic tester's reset button, must never stop a running set
End-of-line "passed"	SPN 628 stays on until the record carries the passed stamp	A station that writes the serial and then aborts must not ship a unit running silently on default trims

Found on the way

- **The SIL engine's load was constant power**, so any overload stalled it in under 3 s, faster than the warning. A real alternator's regulator rolls the voltage off below about 47 Hz, so a resistive load's power falls with speed. The model now has that roll-off (`AVR_KNEE_RPM` , an assumption); scenario 16 settles near 1190 rpm at 30 kW.
- The spec's record size (88 bytes) was wrong: its own fields pack to 84.
- **The final review (20 October)** found an ECU reset accepted on a running set, a lost CAN flow-control frame that silenced channel 2 until power-off, "stopped" reading "turning" after 36 minutes at rest, a partial record clearing SPN 628, USB served from the preemptible background, a save handshake that could lose a write, and the affine CRC-32 key. All seven fixed, each with a test that failed first.

Risks

Risk	What it costs	Answer
The seed-key secret is in every unit's flash	Anyone who reads one unit can compute keys for all	Acceptable for stopping casual writes. Signed images are the answer to tampering. (The first design's CRC-32 key was worse: it is affine, so one sniffed unlock gave keys for every seed. The final review caught it; the key is now SipHash-2-4)
A CAN update at 500 kbit/s	About 30 KiB/s: a 256 KiB image takes about 10 s plus the erase	USB on the prototype
The DSE's under-frequency settings are unknown (U4)	If the DSE ignores DM1 and has no under-frequency trip, nothing stops a set that sits overloaded	In the U4 questions
No rollback	A bad image that passes its CRC needs the previous one flashed	One application slot, by choice
Still to write: the chip layers	Flash, CAN, USB and reset drivers on the STM32G474 and S32K148	The port contract, firmware guide §14 items 10–13

---

17. A simulator anyone can run (20 October)

Asked for: three simulators. (1) The firmware alone. (2) The prototype board fed sensor values and faults over USB, its firmware using them in place of its inputs. (3) The board on a bench rig that plays the engine. Each runs live or from a script, and everything is traced in a browser. This entry is mode 1. The owner's friend uses Windows: "it should be an easy setup for him too".

Question	Answer	Why
Language	**All Rust**: engine model, loop, server	One `.exe`, nothing to install. About 100 times faster than real time, against about 3 times for the Python model, so scripts run in seconds. The same model can later run on the mode 3 bench's microcontroller
The firmware	**Unchanged C**, built in, and loadable from the UI as a `.dll` / `.dylib` / `.so`	The simulator tests the code that ships. A loaded build is refused unless `ecu_abi_version()` and `ecu_io_size()` match
The UI	A web page served by the program itself, on 127.0.0.1 only	Works the same on every OS. `--share` lets a second PC watch, and then refuses firmware uploads, which run code
Scripts	`.seq` text: timed actions, `end`, and `expect` checks	Readable by someone who has never seen the code. A live session is saved in the same syntax, so it replays
The Python SIL	**Kept** as the firmware's gate	It checks what the script language cannot say: the sync edge count, watchdog periods, CAN intervals, the NVM image byte by byte

How it is checked

- The Rust engine model gives the Python model's crank edge times exactly, and speed and rail within 1e-6, over four open-loop profiles.
- The whole loop gives the Python SIL's 5 s start with identical states and speed within 1 rpm.
- All 16 SIL scenarios run as scripts and pass. CI runs them on Windows, macOS and Linux once the branch is pushed.

Found on the way

- The droop scenario claims a 4 % drop, not a speed. Scripts gained one check, `mean change`, for it.
- A DM1 with two faults is sent as a multi-frame BAM. The Python SIL reads only single frames, so a second fault would have gone unseen there. The simulator reassembles them.
- Operating systems hand back an already-loaded library for the same path, so a rebuilt firmware uploaded under the old name would have run the old code. Every load is a fresh copy.

Risks

Risk	Answer
Loaded firmware runs with the user's rights	The UI says so; uploads are refused under `--share`. Only the simulator's own page may send it anything: the final review found that any web site the user visits could POST a library to 127.0.0.1 (no preflight for a plain body), so the Host and Origin headers are checked
The Windows `.exe` needing the VC++ runtime	The C runtime is linked in (`+crt-static`, `/MT`), so nothing is installed
The binary is unsigned	SmartScreen and Gatekeeper warn once; the guide says how to proceed

Risk	Answer
A live session keeps its whole trace in memory	About 300 MB per simulated hour; streaming to disk is the upgrade if long sessions matter
`sin` and `exp` differ in the last bit between OSes	Traces can differ slightly across machines; the checks' bands absorb it

Guide: `simulator/README.md`. Spec and plan: `simulator/docs/`.

18. The Bosch pinout becomes the master (7–8 November)

Asked for: the board had been designed from KOEL's harness diagram (GP3.8703.C4). The owner sent Bosch's terminal diagram for the ECU itself (`T4.435.04.0.PR` , Bosch drawing 0 462 YU4 15D, "MD1CS162 KOEL CPCB4+ 3 cyl"), and the two disagree on power. The owner's answers (7 November): this project replicates the MD1CS162. The Bosch drawing applies to our 3GK550ETA4SR1, because the models it lists share the same sensors and ratings. The "0 281 ..." number on the old ECU is an access code and does not matter. Then: **"use only Bosch pins"**, and **"go with Bosch"**. The pin list is `refs/bosch-pinout-md1cs162.md` , read off a 400 dpi render.

Pin	KOEL diagram (what the board did)	Bosch (what it does now)
21	Permanent battery feed for the electronics	`I_S_WFLS` , the **water-in-fuel** switch, active high
04 / 06	Injector and EGR feeds, plus voltage sense	`V_V_BAT_1R/2R` , the main relay's contact, the board's **only supply**. 04 now also feeds the electronics chain
50	Main relay output, no special role	Closes the board's **own supply** relay
71	Ignition input	Ignition, and the **wake**: it closes the relay in hardware
01 / 02	Unknown, no power ground anywhere	`G_G_BAT1/2` , the **power ground**
09	"Synchronization ground", unassigned	`I_A_SYNC` , an analog input from the GCU
14, 25	Not shown	Diagnosis request; **stop / emergency switch**
CAN	55/77 "channel 1"	87/86 is Bosch's CAN 1 (1000 kbit/s, flashing and UDS); 55/77 is CAN 2 (250 kbit/s, J1939)

Question	Answer	Why
How the board comes up	**Pin 71 turns on a second FET on pin 50** (10k / 22k, 12 V zener): the relay closes in hardware, 04/06 come up, the MCU boots and holds the relay itself	With no permanent feed nothing else can close the relay before the MCU has power. Vgs 5.5 V at 8 V in, 12.0 V on pulse 2a, -0.7 V reversed (`main_relay.cir` A)
Where the electronics feed comes from	**Pin 04**, ahead of its own protection, beside the injector feed	2.95 A of electronics plus 3.8 A RMS of injector stays under the 10 A loom fuse. A blown 04 fuse also stops injection, so losing the ECU with it costs nothing extra

Question	Answer	Why
Pin 50's turn-off clamp	**A TVS to ground** (SMAJ48CA), not a diode to VBAT_PROT	See "Found on the way"
The after-run	Ignition off: the firmware stops the engine, asks for a save, and **releases the relay only once the engine has stopped and the save is written**, at most 60 s	The board drops its own supply. Releasing early would lose the stop's record. The save goes through the 10 s rate limit, and the relay waits for it, so a flapping ignition costs no extra flash writes
The trip loop	**Removed.** The overspeed trip module opens the main relay's coil feed	It had no Bosch pin. Opening the coil drops the ECU, the injectors and the metering unit together, and the GCU sees the CAN go quiet
Pin 25	An operator stop: SHUTDOWN, no DTC, no red lamp, latched until an ignition cycle. **Also in hardware since 10 Nov** (below)	Like the ignition. **Polarity as Bosch draws it** (contact to terminal 31 = stop), the owner's decision on 10 Nov; seen once at first power-up (bring-up step 6)
Pin 21	Amber DTC, SPN 97 FMI 31, after 2 s; clears after 2 s dry	A warning, never a stop. 2 s each way so fuel sloshing past the probe does nothing
Pins 09 and 14	Read and passed to the firmware; **no function yet**	Bosch's drawing names them but does not say what the GCU sends. 09 reads 0–10 V
CAN	Net names kept; the service link moves to **1000 kbit/s**	Renaming nets churns three sheets for nothing. The mapping is written on the connector sheet

Found on the way

- **A sneak path that would have held the relay in forever.** The relay driver had a flyback diode from the drain to VBAT_PROT. That was safe while the coil was fed from the board's own battery feed. Under Bosch's wiring the coil sits on Term. 30, always live. With the relay open, the coil and diode fed the "unpowered" board: 22.8 mA through the coil, 2.7 times its release current, with the board at 9.1 V (`main_relay.cir` B). The relay could never drop out. Bosch marks pin 50 "not equipped with freewheeling diode": the diode belongs in the relay. The netlist check now fails on any diode from pin 50 to a supply.
- **The decoupling check had been passing on nothing.** KiCad 10's `kicad-cli` writes the netlist one element per line. The check's pattern expected KiCad 7's inline form, found no parts, and passed on "0 supply pins". It now parses either form, and fails if it finds none. It checks 50 pins, all decoupled.
- **Two open items closed.** The power ground (since 19 September) and the trip loop's missing pin. The netlist check reports none open.

Proof

- `run_sim.py` : 26/26 blocks. `main_relay` is new; `trip_module_sense` is retired with the loop.
- `check_netlist.py` : connectivity OK, 0 open items. Five new checks: GND on pins 01/02, the 5 A fuse on pin 04, pin 21 an input, no diode from pin 50 to a supply, and two drivers on pin 50 with one woken from pin 71.

- Firmware: the unit tests pass (seven new), as do the boot tests, 16/16 SIL scenarios and the service tool. Scenario 7 now checks that the relay opens 22.6 s after ignition off: after the engine has stopped, and after the stop is saved. `ECU_ABI_VERSION` is 2.
- `check_pcb.py`: footprints, links, nets, classes, widths and DRC pass after `gen_pcb.py`. Schematic parity fails on the Windows machine this was done on, 198 entries, almost all pins with no net. It fails the same on the unchanged board (199), so it is the KiCad build, not this change. It passed on the macOS setup. **Found 9 Nov:** KiCad 10.0.6 drops every pin on an auto-named net (`Net-(...)`, `unconnected-(...)`) from parity, and only those. `check_pcb.py` now checks exactly those pins against KiCad's exported netlist, and parity passes. A pad on the wrong net still fails.

The prototype, brought up to it (8–9 November)

The routed prototype board was updated in place, not placed and routed again (`prototype/gen_board.py`):

- `migrate` records what `sync` cannot decide on its own (`BOSCH_MIGRATE`):
- `/VBAT_IN` becomes `/V_BAT_1R` with its copper. The same rename goes into the committed router inputs `build/power.json` and `build/prototype.dsn`, which the board tests read.
- The trip loop's parts go.
- U301, J1001, J1007 and F101 take the new pad nets. J1007 grows from 6 to 9 positions, with pin 1 kept in place.
- **J1001-1 and -2 are joined by a 2.5 mm F.Cu link.** Both positions are pin 04 now, but their pours had belonged to two nets and did not meet.
- It is safe to run again.
- `sync` **places the 30 new parts.** Grown from the MCU end, the first try put three inputs' front ends 90–150 mm from J1007. Placement now:
- starts from the connector and works inward
- ignores GND and the supply rails when choosing a neighbour
- on a tie, prefers a part placed in the same run

Every input's protection and divider now sits at J1007. - **The three new inputs moved on the MCU:** water in fuel, diagnosis request and the stop switch went from PE7–PE9 to **PD8–PD10** (`stm32_pins.py`). PE7–PE9 are three neighbours mid-way along the bottom row, in the CAN lines' corridor. Routed there, PE8 was walled in, and so were three CAN lines and SENSOR_5V_MON. PD8–PD10 are free pins on the side facing J1007. The firmware reads them through `ecu_io`, so it does not change. - `gnd_stitch` gives each new F.Cu ground pad that reached no via a via to the L2 plane. `pour()` does this for every part, but `sync` did not, so 5 pads lacked one (Q807, Q808, C808, D807, C616). - **D806 is no longer a**

placeholder (`proto_rules.PLACEHOLDER` and `PARITY_OK` are empty). The reference now names the water-in-fuel clamp, a BAV199. - Routing: `finish` and `finish_pass` routed all 42 new connections. A `rip` stage clears a net's tracks before its pins move.

The result: **0 DRC errors, 0 unconnected items, schematic parity clean** (KiCad 10, this machine).

Pin 25 kills the gates in hardware (10 November)

The owner: pin 25 is the operator's force shutdown, pressed when the generator is not behaving properly. So it has to work when the firmware is what is misbehaving. Until now it reached only the firmware.

The change (`hw/gen_sheet_mcu.py`, `stop_kill()`, on both boards): - Q301, the inverter that raises `GATE_KILL`, now takes its gate from `~RESET` through a 100k (R307; R314 on the prototype). - A 2N7002BK (Q302) with its gate on `SW_STOP` pulls that gate low while the button is pressed. `GATE_KILL` rises and every injector, metering and EGR gate drops, with no firmware in the path. - `~RESET` stays at 3.0 V, above the MCU's 2.31 V threshold, so the MCU runs on, sees the stop and latches SHUTDOWN. Releasing the button does not restart injection.

Proven by: - `sim/blocks/stop_kill.cir`, at the worst-high FET threshold: - pressed: 11.9 V on `GATE_KILL` - a 1 kΩ contact still kills; 47 kΩ of harness leakage does not; it trips at 10.8 kΩ - 0.9 ms from press to kill - `check_netlist` STOP_KILL, on both boards.

Polarity: the drawing's (contact to terminal 31 when pressed), chosen by the owner on 10 Nov. If the machine shows a normally-closed contact, this path and the firmware both invert.

Still open

- **The Rust simulator** mirrors `ecu_io`. Its fields and ABI are updated, but there was no Rust toolchain on this machine to build it. CI will build it, and so will a `cargo` on any of the three OSes. The committed Windows `.exe` is the old build until then.
- ~~At the machine: whether the fitted main relay carries its own suppressor.~~ Closed 10 Nov. Pin 50 drives the main relay and 69 the buzzer relay (owner). Neither needs to know about suppressors:
- Pin 50's TVS works with one in the relay or without.
- Pin 69's flyback diode (D811) goes to the board's supply. That is safe because the buzzer coil is fed from the same main-relay line, as Bosch draws it, so there is no sneak path when the relay opens.
- **Pin 25's polarity: as Bosch draws it** (owner, 10 Nov 2026). The switch is the external stop/emergency button, and pressing it connects pin 25 to terminal 31. It is no longer a question for the machine; first power-up shows it (bring-up step 6).

- **Pins 09 and 14 (owner, 10 Nov):** they stay as an analog input (09, 0–10 V into the ADC) and a digital switch input (14, contact to ground), at their current ratings. What they do is configured in firmware later. Until then the firmware reads both and acts on neither, so neither can change how the engine runs.

19. The boost cannot run away; heat and coating (12 November)

Asked for: the owner asked what still bothered us, then for fixes for the 100 V boost and for heat: **"fix this"**.

Found: nothing stopped the boost in hardware. The TPS40210 holds 99.7 V through 2 × 499 k into FB. If either resistor opens (a cracked 1206, a lifted joint), FB reads 0 V and the controller runs at maximum duty. Its DIS/EN pin was tied to "on" through 10 k, and nothing else stopped the rail before a 150 V part failed (FET, diode or the 160 V capacitor stack). The production board doesn't measure the rail at all.

Question	Answer	Why
The cutoff	**A 110 V zener (BZG03C110-HM3) from BOOST_100V, through 470 Ω, into DIS/EN**; the pulldown goes from 10 k to 470 Ω	DIS/EN is an active-high shutdown, 0.7–1.3 V, rated 10 V (TI SLVS861F §6.1, §6.5, §7.3.7). 470/470 puts the zener at 1.5–2.8 mA at the trip, near its 5 mA test current, so its 104–116 V range applies. **Trip 105.0–119.3 V** over both corners; with feedback open the rail is held at 119.3 V, dissipating 0.66 W in the zener (`run_sim.py boost_ovp`)
Above the trip	**A 5.1 V zener (BZX84-C5V1-Q) on DIS/EN**	Holds the pin at 6.2 V with the rail at 160 V, under its 10 V rating
The MCU's stop	**`BOOST_DIS` : PTD15 (production), PD14 (prototype), through 100 Ω and a BAT54Q into DIS/EN**	2.1 V on EN at the MCU's 2.9 V minimum high, 4.5 mA. The Schottky keeps a low or Hi-Z pin from loading the zener's divider. Hi-Z at reset leaves the boost running, as before
Firmware	**Above 106 V for 20 ms: `boost_dis` , SHUTDOWN, red DTC SPN 520192 FMI 0**, latched to the next ignition cycle. The boost is also off in SHUTDOWN	With the boost off, 12V_GATE collapses and nothing can inject, so the stop is made explicit. 520192 is the first proprietary SPN: J1939 has none for an injector boost rail. Production has no rail channel (`v_boost_rail 0`), so there only the zener guards it
Passives at the fault peak	**BOOST peak 104 → 120 V** in `assign_passives.py`	Nothing changes size: the 0805s on the rail are rated 150 V

Heat. The hot parts were sized against a housing the prototype doesn't have: Q104 at 139 °C junction with the EGR valve stalled at 6.4 A, on a 105 °C board. The firmware had no EGR control loop, so it had no stall limit either. **Now it has a stall guard.** Over 5 A for 1.5 s, the bridge coasts (`egr_off` : the valve springs shut) for 5 s and an amber DTC is raised: SPN 2791 (EGR valve control), FMI 6. The third stall holds it off until the next ignition cycle. EGR is never a reason to stop the engine. Bring-up step 7 adds a 10-minute full-load heat check with a 100 °C limit.

Spacing. Every 100 V net keeps 0.6 mm. That is IPC-2221B's B2 row for an uncoated external layer up to 150 V, so it's fine on a bench. Raising it to 1.0 mm would fail hundreds of places, and many of them are inside the FET packages themselves, where layout can't help. So **the board is conformal-coated before it goes on the engine** (acrylic or silicone, connectors and test points masked). The bring-up guide says so, and so does the layout guide for production.

The prototype, brought up to it. `migrate boost_ovp` refits R403 (10 k → 470 Ω) and U301 (PD14 now drives `BOOST_DIS`); `sync` places the five new parts beside U401. Sync dropped R437 inside the ISNS_BOOST pour (Q401's source to the 62 mΩ sense resistor), and its first routing cut that pour in two (`test_power_pours_connect`). R437 was moved to the nearest clear spot, left of the pour, and the four nets were re-routed in one pass. 26/26 prototype tests pass, with 0 DRC errors, nothing unconnected, and the fab files regenerated.

ABI 3. `ecu_io` gains `v_boost_rail`, `i_egr`, `boost_dis` and `egr_off`. The Python SIL and the Rust simulator's copies are updated. The simulator wasn't rebuilt (owner: "no simulator").

20. The application in Simulink and Stateflow, with generated code (13 November)

Asked for: "a complete state flow model in MATLAB Simulink that is able to generate code, and the new generated code we should be able to flash in our ECU". The owner left the how to us ("do the best you can"). Design: `docs/superpowers/specs/2026-11-13-simulink-model-design.md`.

Question	Answer	Why
What the models own	**The application layer**: the engine state machine, governor, rail control, the two calibration tables, the injector law, the trip decision and the five timed guards. Twelve models	That is the logic engineers tune and review. Tooth decoding, J1939, UDS, storage and the bootloader are event- and byte-driven platform code, already tested, and stay C
How they plug in	**Drop-in components** behind today's functions (`gov_step`, `mode_cmd` ...), selected by `ECU_MODEL`	One model for the whole tick would either slow the per-tooth trips (overspeed, sync) to the tick or split one state machine between C and model
The state machine	**A Stateflow chart driven by commands** (IGN_ON, TOOTH, RUN_OK, TRIP, RESET)	The tooth interrupt and the tick both change the state. `ecu.c` had five places that wrote it; all five now ask the chart
The guards	**Five small charts**, each its own model (`mode.c` holds the hand-written reference, moved out of `ecu.c` unchanged)	They reset at different times (water in fuel only at power-up) and run at different points in the tick
Where constants come from	**The C headers and tables**, read by `model/ecu_params.m`	A calibration typed in two places drifts. The `.slx` files are built by scripts, like the schematics

Question	Answer	Why
Code settings	ERT, reusable functions, inlined parameters, single precision, ARM Cortex-M, shared utilities	Runs on the STM32G474 and the S32K148 alike; no `double` on a single-precision FPU
Which is the reference	**Both builds, for now**	The hand-written modules stay until the generated build has run on the engine

The proof. Every unit test and all 16 SIL scenarios pass on the generated build. A back-to-back test (`model/test/run_b2b.sh`) runs each generated model against its hand-written original on millions of random inputs; the six charts are run 3 million random ticks each, reaching every outcome (84 boost trips, 143 EGR stalls, 46 after-runs, every engine state). They agree exactly, apart from one or two units in the last place in the two tables, where Simulink orders its interpolation arithmetic differently. The closed-loop model (`ecu_app` : the real component models with a mean-value engine) starts, holds 1500 rpm, takes 20 kW with a 1444 rpm low and stops on pin 25 in Simulink (`model/test/sim_ecu_app.m`).

Found on the way: - The governor first came out one unit in the last place off. The compiler, not the model, did it: it fused `set + RAMP*dt` into one multiply-add in the hand-written C. With `-ffp-contract=off` both agree to the bit. - Simulating the charts in Simulink caught locals read before they were first written, which the C tests could not see. They start at zero in the generated C either way, but every chart now says so. - `python -m ziglang cc` swallows the compiler's error messages on this machine. Two silent failures (a duplicated helper across two models, a missing source file) came from that; `zig.exe` called directly prints them.

What it does not do yet: run on the board. The generated code is part of the firmware image and is flashed with it, over SWD or the UDS bootloader. But the STM32 chip layer (ADC, timers, CAN, USB, flash) is still unwritten, for either build.

14 November, closing what still bothered us (the owner: "can we fix any of these"): - **The boost's silent clamp, and a dead boost.** The hardware zener can hold the rail at 105 V, below the firmware's 106 V trip, and nothing said so. A dead boost showed only as "won't start". Now: held over 104 V for 2 s raises amber SPN 520192 FMI 16; under 80 V for 0.5 s while cranking or running raises amber FMI 18. Neither stops the engine. Both are in the C and in a new chart, `ecu_boostm` ; the back-to-back test agrees on every tick. - **The state machine owns its conditions.** `ecu.c` used to decide when the ignition counted and when an ignition cycle had finished; it now sends TICK every millisecond and the chart decides both. The Simulink closed loop lost the copy of that logic it carried. - **One command checks everything:** `./verify.sh` . - **A test that passed by luck.** The hour-counter test depended on the edge

generator's leftover fraction of a microsecond from earlier tests: that bias decided whether the governor's integral drifted up or down over a simulated hour. Adding a test before it flipped it. Each test now starts the edge clock clean.

Where these came from

Nothing here was found by reading the schematic. Each one surfaced when a part number was chosen and its datasheet was checked against what the drawing assumed — which is the same shape as every earlier finding in this project: a document and an executable check disagreeing, with nothing comparing them until someone did.